

Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- ▶ Recurrent Neural Networks
- ▶ RNNs as Language Models
- ▶ RNNs for other NLP tasks
- ▶ Stacked and Bidirectional RNN architectures
- ▶ The LSTM
- ▶ Summary: Common RNN NLP Architectures

Recurrent Neural Network (RNN)

- ▶ RNNs have a mechanism that deals directly with the sequential nature of language, allowing them to handle the temporal nature of language without the use of arbitrary fixed-sized windows. The recurrent network offers a new way to represent the prior context, in its recurrent connections, allowing the model's decision to depend on information from hundreds of words in the past

Recurrent neural network (RNN)

- ▶ A recurrent neural network (RNN) is any network that contains a cycle within its network connections, meaning that the value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input.
- ▶ we consider a class of recurrent networks referred to as Elman Networks (Elman, 1990) or simple recurrent net. These networks are useful in their own right and serve as the basis for more complex approaches like the Long Short-Term Memory (LSTM) networks.

RNN

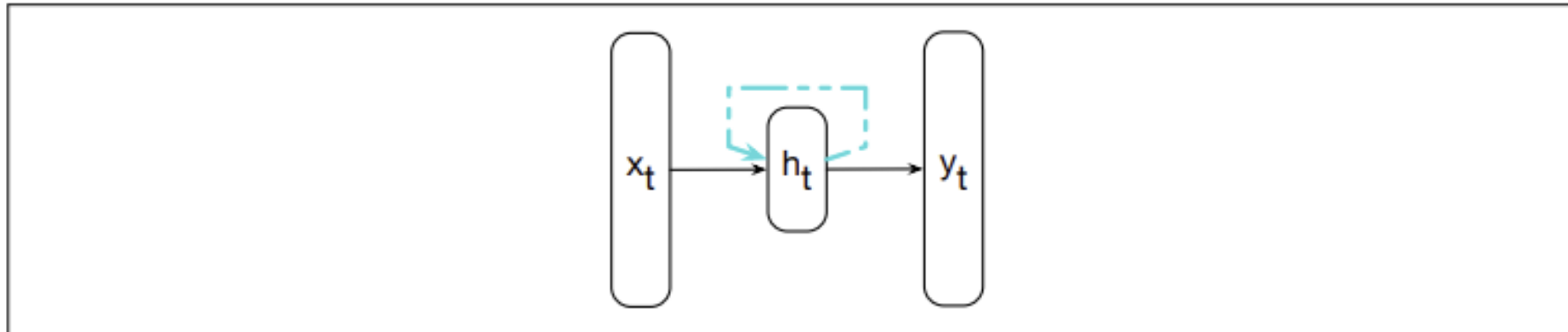


Figure 9.1 Simple recurrent neural network after [Elman \(1990\)](#). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

RNN

- ▶ As with ordinary feedforward networks, an input vector representing the current input, X_t , is multiplied by a weight matrix and then passed through a non-linear activation function to compute the values for a layer of hidden units. This hidden layer is then used to calculate a corresponding output, Y_t . Here, sequences are processed by presenting one item at a time to the network. We'll use subscripts to represent time, thus X_t will mean the input vector X at time t . The key difference from a feedforward network lies in the recurrent link shown in the figure with the dashed line. This link augments the input to the computation at the hidden layer with the value of the hidden layer from the preceding point in time.

RNN

- ▶ The hidden layer from the previous time step provides a form of memory, or context, that encodes earlier processing and informs the decisions to be made at later points in time. Critically, this approach does not impose a fixed-length limit on this prior context; the context embodied in the previous hidden layer can include information extending back to the beginning of the sequence.
- ▶ Adding this temporal dimension makes RNNs appear to be more complex than non-recurrent architectures. But in reality, they're not all that different. Given an input vector and the values for the hidden layer from the previous time step, we're still performing the standard feedforward calculation.

RNN

- ▶ The most significant change lies in the new set of weights U , that connect the hidden layer from the previous time step to the current hidden layer. These weights determine how the network makes use of past context in calculating the output for the current input. As with the other weights in the network, these connections are trained via backpropagation.

Inference in RNNs

- ▶ Forward inference (mapping a sequence of inputs to a sequence of outputs) in an RNN is nearly identical to what we've already seen with feedforward networks. To compute an output Y_t for an input X_t , we need the activation value for the hidden layer h_t . To calculate this, we multiply the input X_t with the weight matrix W , and the hidden layer from the previous time step h_{t-1} with the weight matrix U . We add these values together and pass them through a suitable activation function, g , to arrive at the activation value for the current hidden layer, h_t . Once we have the values for the hidden layer, we proceed with the usual computation to generate the output vector

$$h_t = g(Uh_{t-1} + Wx_t) \quad (9.1)$$

$$y_t = f(Vh_t) \quad (9.2)$$

Inference in RNNs

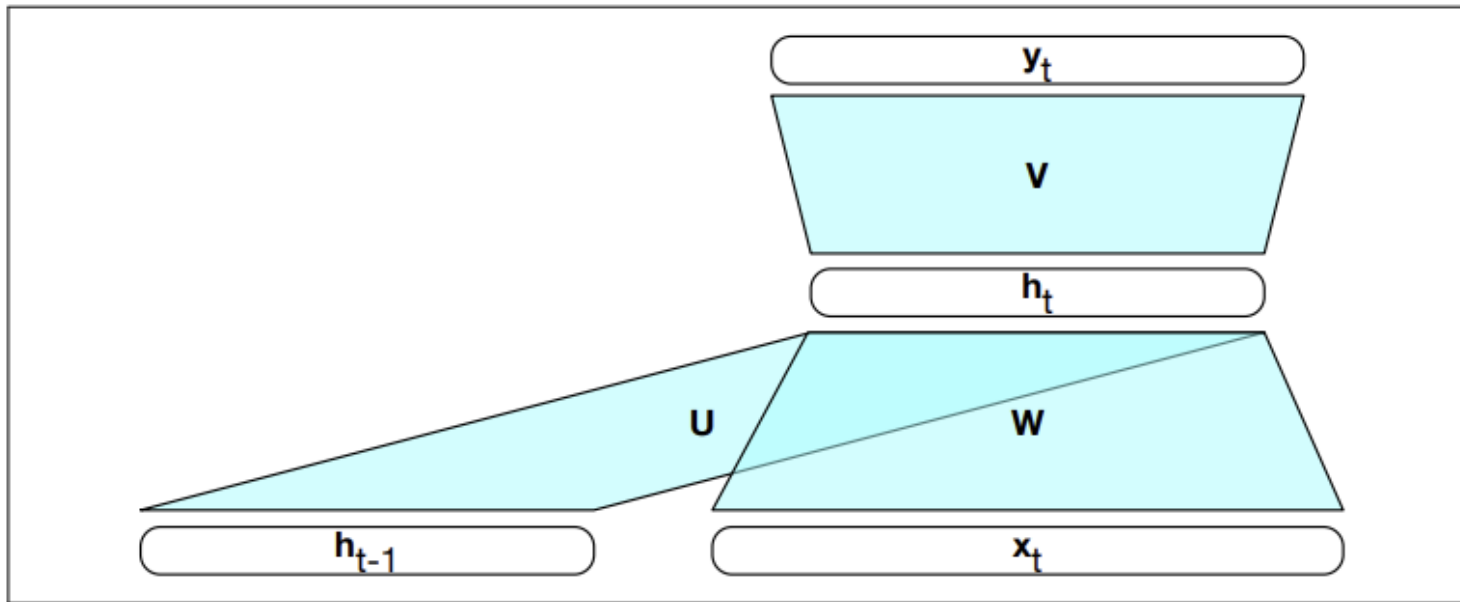


Figure 9.2 Simple recurrent neural network illustrated as a feedforward network.

Inference in RNNs

- ▶ be careful about specifying the dimensions of the input, hidden and output layers, as well as the weight matrices to make sure these calculations are correct. Let's refer to the input, hidden and output layer dimensions as d_{in} , d_h , and d_{out} respectively. Given this, our three parameter matrices are: $W \in \mathbb{R}^{d_h \times d_{in}}$, $U \in \mathbb{R}^{d_h \times d_h}$, and $V \in \mathbb{R}^{d_{out} \times d_h}$. In the commonly encountered case of soft classification, computing Y_t consists of a softmax computation that provides a probability distribution over the possible output classes.

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) \quad (9.3)$$

- ▶ The fact that the computation at time t requires the value of the hidden layer from time $t - 1$ mandates an incremental inference algorithm that proceeds from the start of the sequence to the end as illustrated in Fig. 9.3

```
function FORWARDRNN( $\mathbf{x}$ ,  $network$ ) returns output sequence  $\mathbf{y}$ 
```

```
   $\mathbf{h}_0 \leftarrow 0$ 
```

```
  for  $i \leftarrow 1$  to LENGTH( $\mathbf{x}$ ) do
```

```
     $\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$ 
```

```
     $\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$ 
```

```
  return  $\mathbf{y}$ 
```

Figure 9.3 Forward inference in a simple recurrent network. The matrices $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$ are shared across time, while new values for $\mathbf{h}$ and $\mathbf{y}$ are calculated with each time step.

Training

- ▶ As with feedforward networks, we'll use a training set, a loss function, and backpropagation to obtain the gradients needed to adjust the weights in these recurrent networks.
- ▶ As shown in Fig. 9.2, we now have 3 sets of weights to update: W , the weights from the input layer to the hidden layer, U , the weights from the previous hidden layer to the current hidden layer, and finally V , the weights from the hidden layer to the output layer.

Training

- ▶ Fig. 9.4 highlights two considerations that backpropagation in feedforward networks. First, to compute the loss function for the output at time t we need the hidden layer from time $t - 1$. Second, the hidden layer at time t influences both the output at time t and the hidden layer at time $t + 1$ (and hence the output and loss at $t + 1$). It follows from this that to assess the error accruing to h_t , we'll need to know its influence on both the current output as well as the ones that follow.

Training

- ▶ Tailoring the backpropagation algorithm to this situation leads to a **two-pass algorithm for training the weights in RNNs**.
- ▶ In the first pass, **we perform forward inference**, computing h_t , y_t , accumulating the loss at each step in time, saving the value of the hidden layer at each step for use at the next time step.
- ▶ In the second phase, **we process the sequence in reverse**, computing the required gradients as we go, computing and saving the error term for use in the hidden layer for each step backward in time. **This general approach is commonly referred to as backpropagation through time** (Werbos 1974, Rumelhart et al. 1986, Werbos 1990)

RNNs as Language Models

- ▶ For example, if the preceding context is “Thanks for all the” and we want to know how likely the next word is “fish” we would compute:

$$P(\textit{fish}|\textit{Thanks for all the})$$

Language models give us the ability to assign such a conditional probability to every possible next word, giving us a distribution over the entire vocabulary. We can also assign probabilities to entire sequences by combining these conditional probabilities with the chain rule:

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i|w_{<i})$$

RNNs as Language Models

- ▶ RNN language models (Mikolov et al., 2010) process the input sequence one word at a time, attempting to predict the next word from the current word and the previous hidden state. RNNs thus don't have the limited context problem that n-gram models have, or the fixed context that feedforward language models have, since the hidden state can in principle represent information about all of the preceding words all the way back to the beginning of the sequence

Forward Inference in an RNN language model

- ▶ The input sequence $X = [x_1; \dots; x_t; \dots; x_N]$ consists of a series of words each represented as a one-hot vector of size $|V| \times 1$, and the output prediction, y , is a vector representing a probability distribution over the vocabulary. At each step, the model uses the word embedding matrix E to retrieve the embedding for the current word, and then combines it with the hidden layer from the previous step to compute a new hidden layer. This hidden layer is then used to generate an output layer which is passed through a softmax layer to generate a probability distribution over the entire vocabulary. That is, at time t :

$$\mathbf{e}_t = E\mathbf{x}_t \quad (9.4)$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \quad (9.5)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) \quad (9.6)$$

Forward Inference in an RNN language model

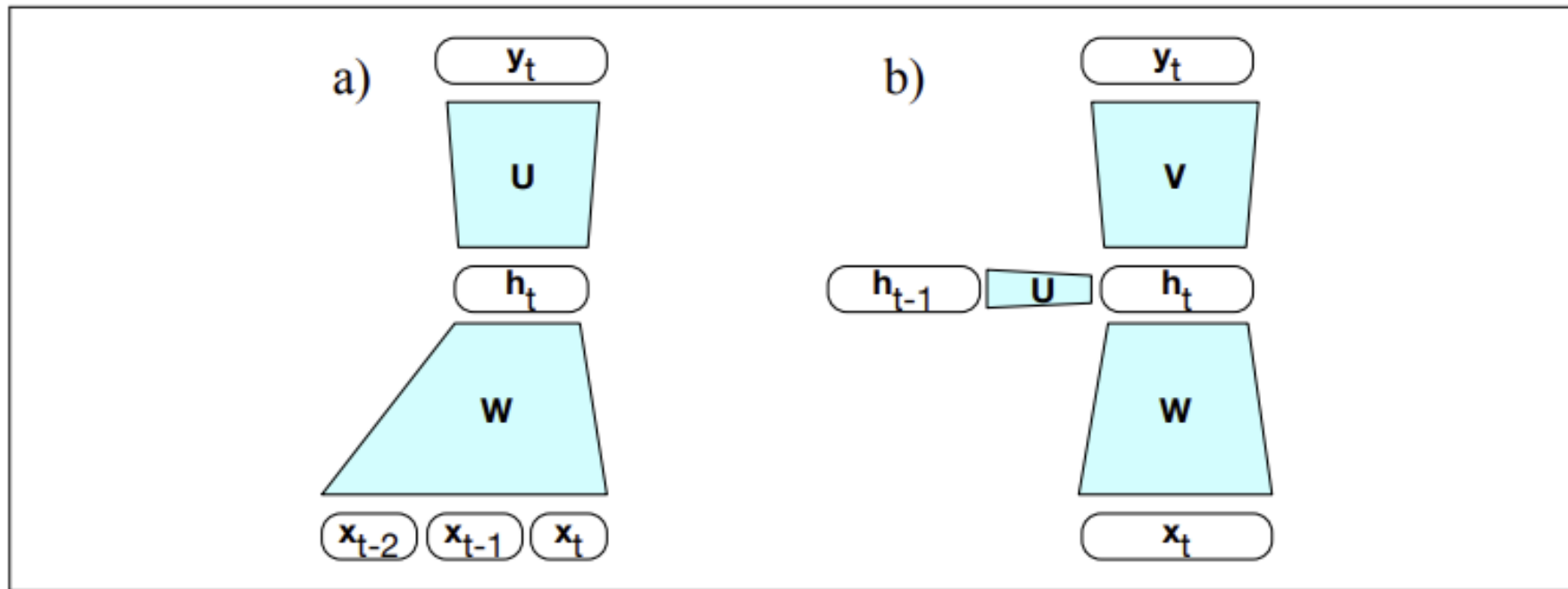


Figure 9.5 Simplified sketch of (a) a feedforward neural language model versus (b) an RNN language model moving through a text.

Forward Inference in an RNN language model

- ▶ The vector resulting from Vh can be thought of as a set of scores over the vocabulary given the evidence provided in h . Passing these scores through the softmax normalizes the scores into a probability distribution. The probability that a particular word l in the vocabulary is the next word is represented by $y_t[k]$, the k th component of y_t :

$$P(w_{t+1} = k | w_1, \dots, w_t) = \mathbf{y}_t[k] \quad (9.7)$$

Forward Inference in an RNN language model

The probability of an entire sequence is just the product of the probabilities of each item in the sequence, where we'll use $Y_i[w_i]$ to mean the probability of the true word W_i at time step i .

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1}) \quad (9.8)$$

$$= \prod_{i=1}^n \mathbf{y}_i[w_i] \quad (9.9)$$

Training an RNN language model

- ▶ To train an RNN as a language model, we use the same self-supervision (or self-training) algorithm we saw earlier. we take a corpus of text as training material and at each time step t ask the model to predict the next word. We call such a model self-supervised because we don't have to add any special gold labels to the data; Recall that the cross-entropy loss measures the difference between a predicted probability distribution and the correct distribution.

$$L_{CE} = - \sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w] \quad (9.10)$$

Training an RNN as language model

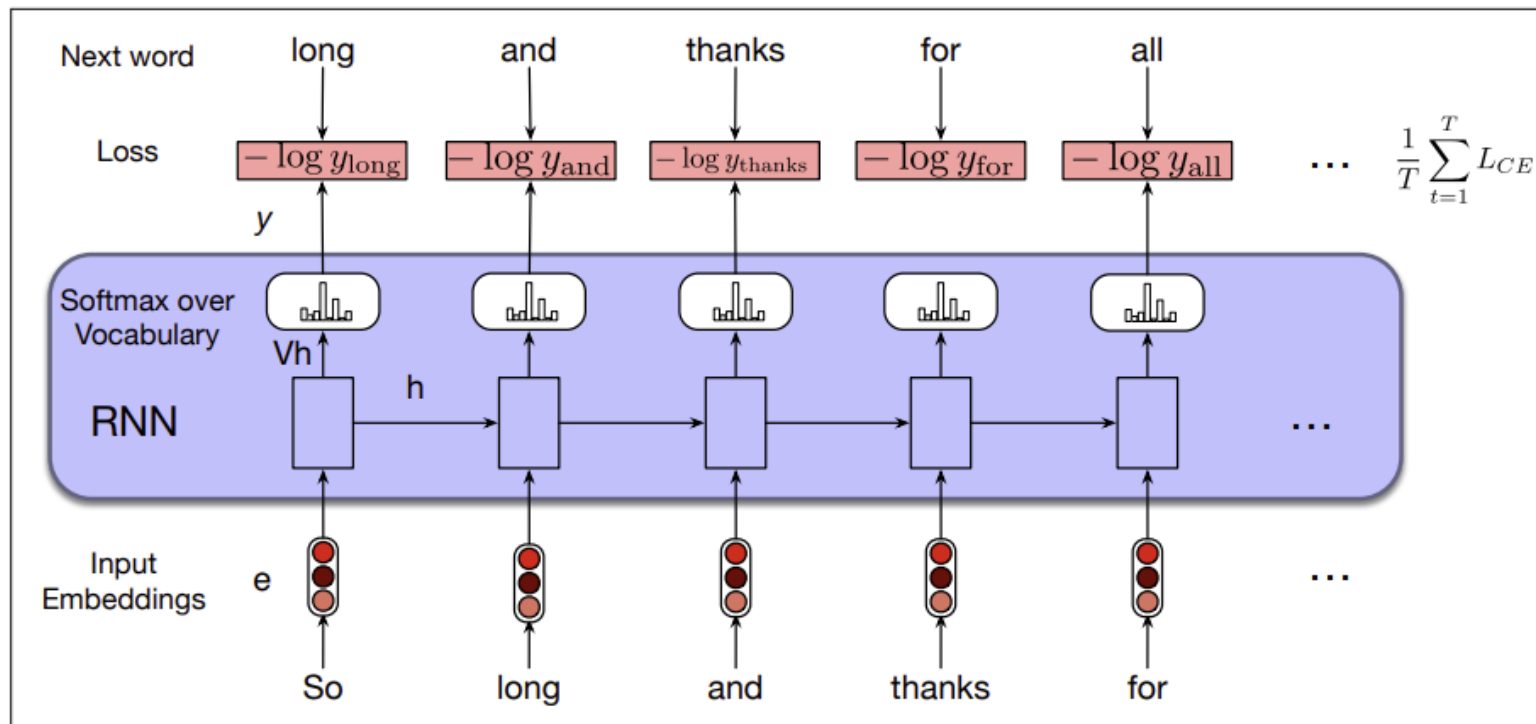


Figure 9.6 Training RNNs as language models.

In the case of language modeling, the correct distribution Y_t comes from knowing the next word. This is represented as a one-hot vector corresponding to the vocabulary

Training an RNN as language model

- ▶ where the entry for the actual next word is 1, and all the other entries are 0. Thus, the cross-entropy loss for language modeling is determined by the probability the model assigns to the correct next word. So at time t the CE loss is the negative log probability the model assigns to the next word in the training sequence.

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}] \quad (9.11)$$

Weight Tying

The input embedding matrix E and the final layer matrix V , which feeds the output softmax. The columns of E represent the word embeddings for each word in the vocabulary learned during the training process with the goal that words that have similar meaning and function will have similar embeddings. And, since the length of these embeddings corresponds to the size of the hidden layer d_h , the shape of the embedding matrix E is $d_h \times |V|$.

Weight Tying

- ▶ The final layer matrix V provides a way to score the likelihood of each word in the vocabulary given the evidence present in the final hidden layer of the network through the calculation of Vh . This results in dimensionality $|V| \times d_h$. That is, the rows of V provide a second set of learned word embeddings that capture relevant aspects of word meaning. **Weight tying is a method that dispenses with this redundancy and simply uses a single set of embeddings at the input and softmax layers.** That is, we dispense with V and use E in both the start and end of the computation.

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t \quad (9.12)$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \quad (9.13)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{E}^T \mathbf{h}_t) \quad (9.14)$$

RNNs for other NLP tasks

- ▶ Apply RNN to three types of NLP tasks: **sequence classification** tasks like **sentiment analysis** and **topic classification**, sequence labeling tasks like part-of-speech tagging, and text generation tasks, including with a new architecture called the encoder-decoder.
- ▶ **Sequence Labeling** : the network's task is to assign a label chosen from a small fixed set of labels to each **element of a sequence**, like the **part-of-speech tagging** and **named entity recognition** tasks.

- In an RNN approach to sequence labeling, **inputs are word embeddings** and the **outputs are tag probabilities generated by a softmax layer** over the given tagset, as illustrated in Fig. 9.7.

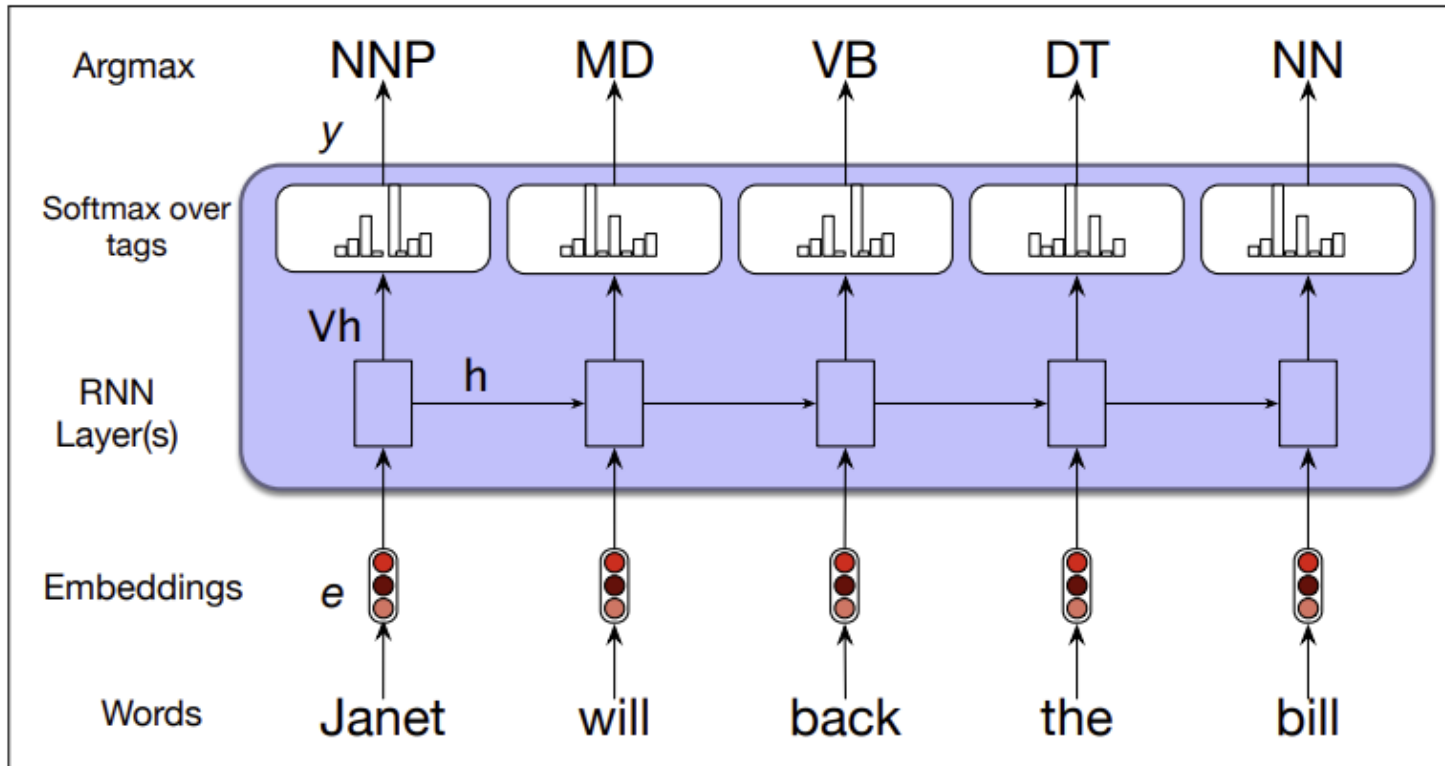


Figure 9.7 Part-of-speech tagging as sequence labeling with a simple RNN. Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

Parts of speech tagging

- ▶ In this figure, the inputs at each time step are pretrained word embeddings corresponding to the input tokens. The RNN block is an abstraction that represents an unrolled simple recurrent network consisting of an input layer, hidden layer, and output layer at each time step, as well as the shared U , V and W weight matrices that comprise the network.

Parts of speech tagging

- ▶ The outputs of the network at each time step represent the distribution over the POS tagset generated by a softmax layer. To generate a sequence of tags for a given input, we run forward inference over the input sequence and select the most likely tag from the softmax at each step. Since we're using a softmax layer to generate the probability distribution over the output tagset at each time step.

RNNs for Sequence Classification

- ▶ This is the set of tasks commonly called text classification, like sentiment analysis or spam detection, in which we classify a text into two or three classes as well as classification tasks with a large number of categories, like document-level topic classification, or message routing for customer service applications.
- ▶ To apply RNNs in this setting, we pass the text to be classified through the RNN a word at a time generating a new hidden layer at each time step. We can then take the hidden layer for the last token of the text, h_n , to constitute a compressed representation of the entire sequence. We can pass this representation h_n to a feedforward network that chooses a class via a softmax over the possible classes

RNNs for Sequence Classification

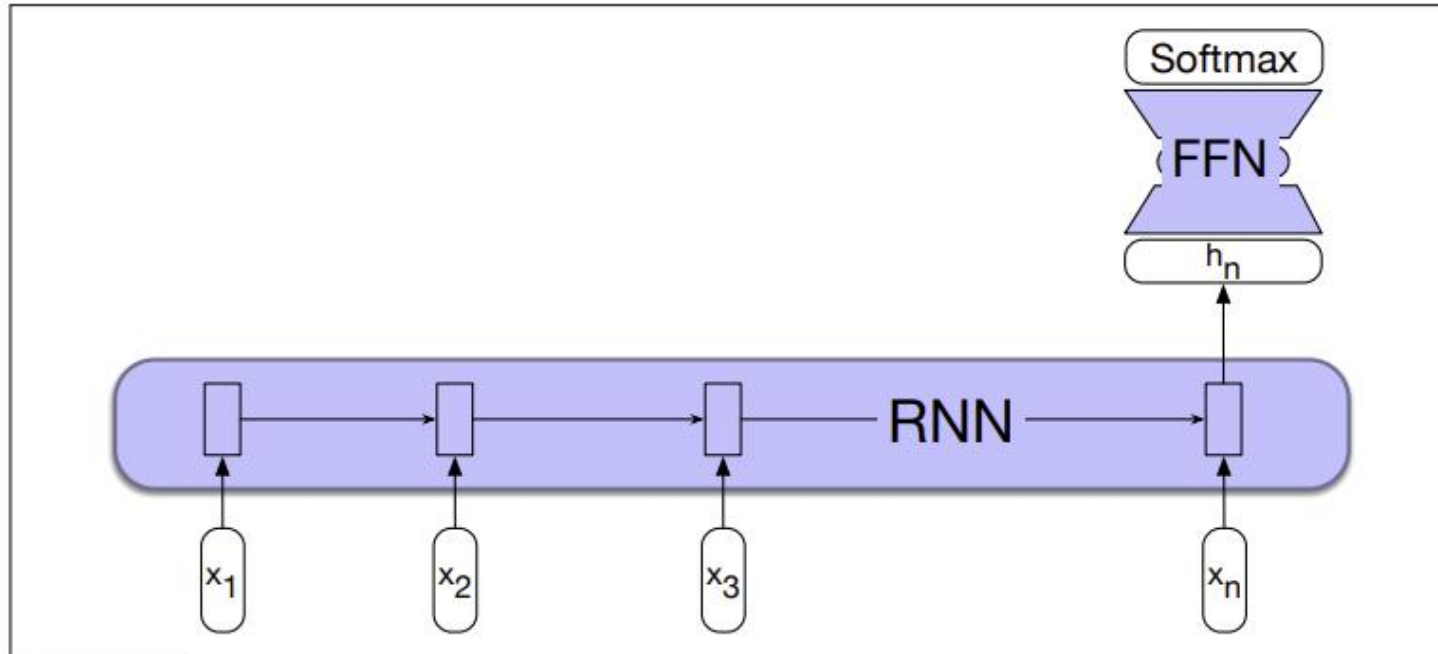


Figure 9.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

RNNs for Sequence Classification

- ▶ Note that in this approach **we don't need intermediate outputs for the words in the sequence preceding the last element**. Therefore, there are no loss terms associated with those elements. Instead, the loss function used to train the weights in the network is based entirely on the final text classification task. The output from the softmax output from the feedforward classifier together with a cross-entropy loss drives the training.

Generation with RNN-Based Language Models

- ▶ Text generation is of enormous practical importance, part of tasks like question answering, machine translation, text summarization, grammar correction, story generation, and conversational dialogue; any task where a system needs to produce text, conditioned on some other text.
- ▶ Shannon (Shannon, 1951) and the psychologists George Miller and Jennifer Selfridge (Miller and Selfridge, 1950). We first randomly sample a word to begin a sequence based on its suitability as the start of a sequence. We then continue to sample words conditioned on our previous choices until we reach a pre-determined length, or an end of sequence token is generated.

Generation with RNN-Based Language Models

- ▶ Technically an autoregressive model is a model that predicts a value at time t based on a linear function of the previous values at times $t - 1$, $t - 2$, and so on. Although language models are not linear (since they have many layers of non-linearities), we loosely refer to this generation technique as autoregressive generation since the word generated at each time step is conditioned on the word selected by the network from the previous step. Fig. 9.9 illustrates this approach. In this figure, the details of the RNN's hidden layers and recurrent connections are hidden within the blue block

Generation with RNN-Based Language Models

- ▶ This simple architecture underlies state-of-the-art approaches to applications such as machine translation, summarization, and question answering.

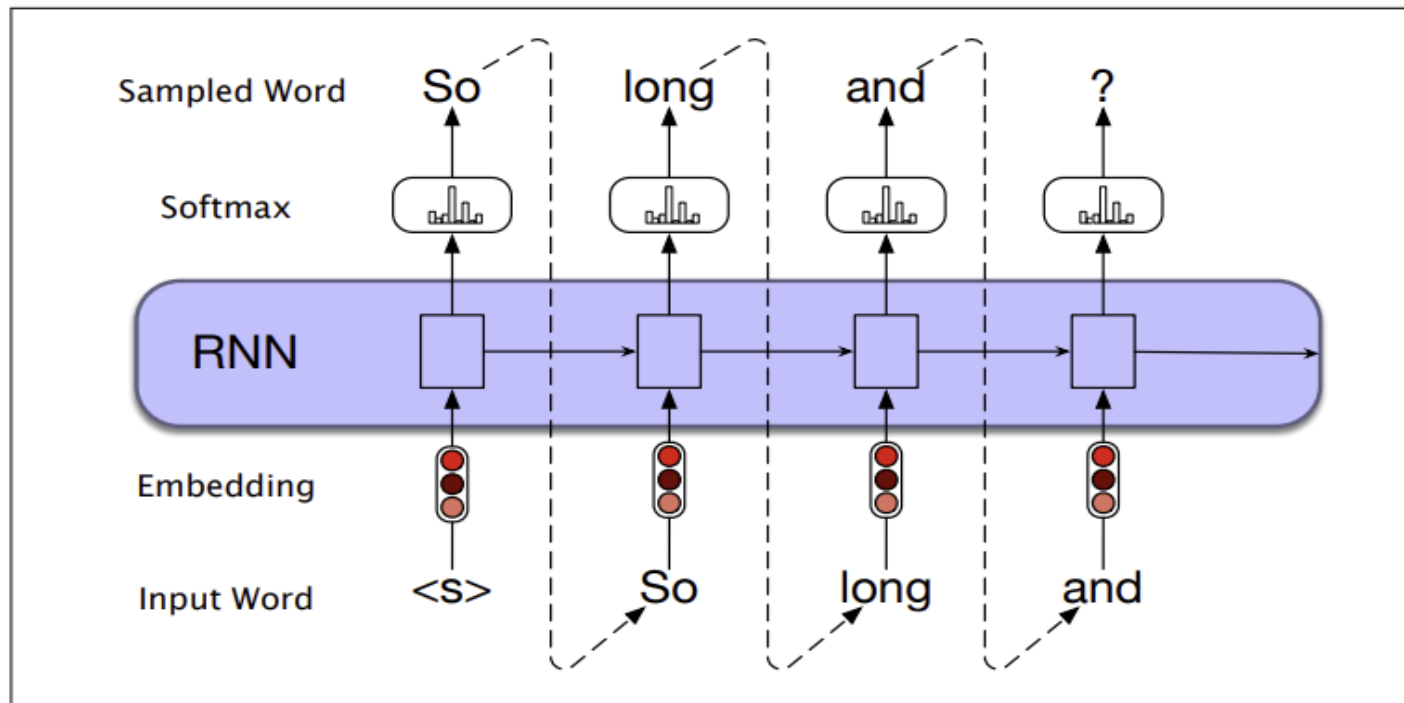


Figure 9.9 Autoregressive generation with an RNN-based neural language model.

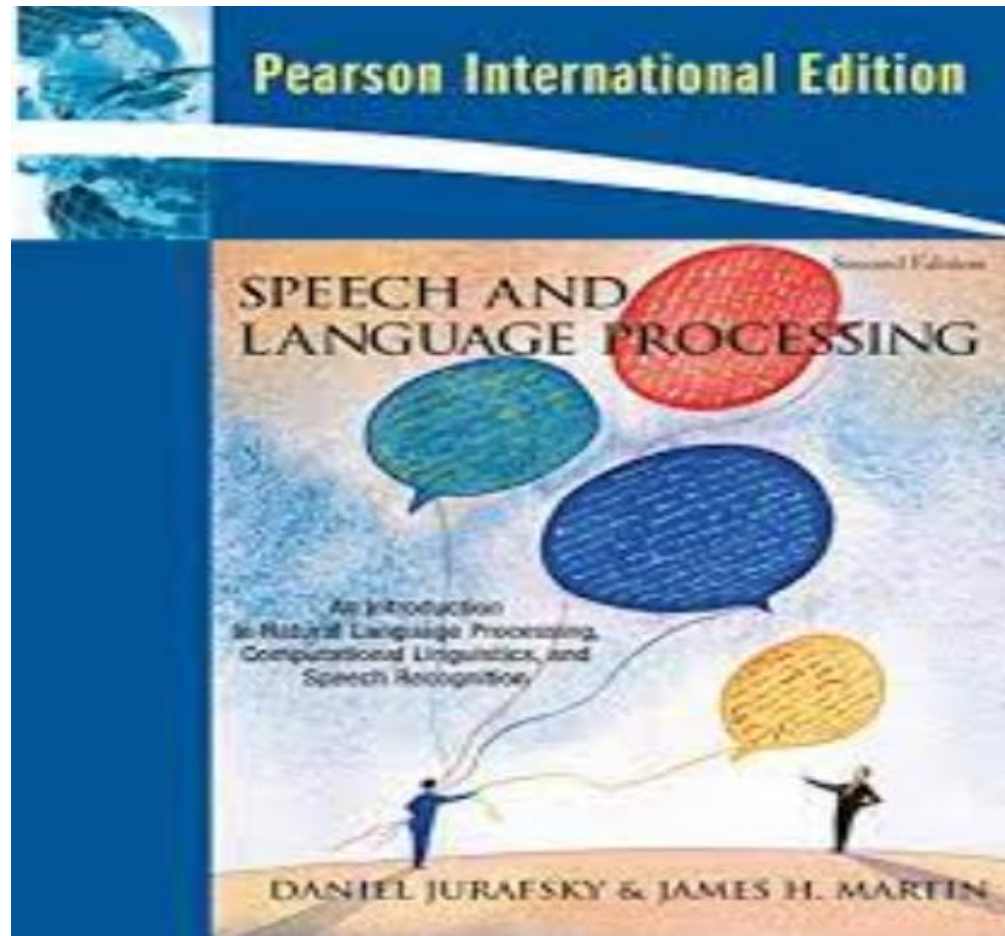
Summary

- ▶ Structure :RNNs have layers of neurons, but they include recurrent connections that allow information to persist over time. In RNNs, neurons have connections to themselves, allowing them to store information about previous inputs
- ▶ Function: RNNs are designed to handle sequential data, where the order of inputs matters. They excel at tasks such as natural language processing (NLP), time series analysis, speech recognition, and handwriting recognition.
- ▶ Memory: RNNs have memory due to their recurrent connections. They can maintain information about past inputs and use it to influence the processing of future inputs. This makes RNNs well-suited for tasks involving sequences or time series data.

Summary

- ▶ Training: RNN: RNNs are trained using a variation of backpropagation called **backpropagation through time** (BPTT), which accounts for the temporal nature of the data. However, training RNNs can be challenging due to issues like vanishing gradients, where gradients become too small to update the weights effectively.
- ▶ RNNs are particularly useful for tasks involving sequential data, such as language translation, sentiment analysis, and generating text.

Reference



Chapter 9

Question



Thank you

